Approach summary

Wenqi Deng
2400013097@stu.pku.edu.cn

1 Introduction

We cast subgraph scheduling as a search problem. Each state is defined by three components: the set of already covered operations, the tensors stored in slow memory, and the tensors currently resident in fast memory. The start state contains no covered operations, with all graph inputs initially stored in slow memory and no tensor resident in fast memory. The goal state is reached when all operations have been covered and all graph outputs reside in slow memory. A state transition corresponds to selecting a feasible subgraph execution: all required inputs must either already be in fast memory or be loadable from slow memory, and the resulting execution must satisfy the fast-memory capacity constraint.

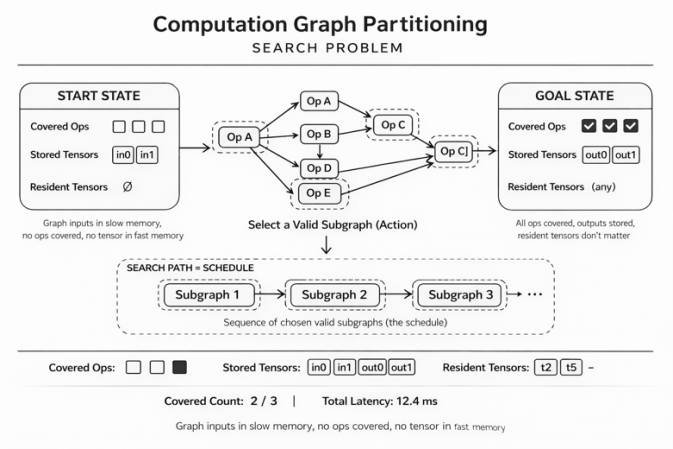

Figure 1: Search model

2 Method

2.1 Beam search

We use beam search to avoid making purely greedy decisions during scheduling. At each search step, every search state in the current beam is expanded by enumerating the feasible next subgraphs. The new states are sorted and the top-ranked states are kept for the next beam. By maintaining multiple promising partial scheduling, beam search reduces the risk of getting trapped in poor optima.

2.2 Candidate Search

When enumerating the next feasible subgraph, we first propose candidate subgraphs, which are sets of operations that are likely to form a valid subgraph. We generate candidate subgraphs from several structural templates, including single operators, linear pointwise chains, pointwise cones, recompute cones and MatMul-Pointwise chain. This reduces the search space by opting out many illegal schedules.

Author's Contact Information: Wenqi Deng, 2400013097@stu.pku.edu.cn.

2.3 Candidate evaluation

For each candidate subgraph, we enumerate possible granules and check whether it leads to execution that fit within fast memory capacity.

For a single MatMul candidates and MatMul–Pointwise chains, we first enumerate spatial tile sizes (w, h). For each spatial choice, instead of exhaustively enumerating all possible split-k values, we try k from large to small and accept the first feasible value. Since the largest feasible k usually gives the lowest compute cost under the same spatial tile, this greatly reduces the search cost without sacrificing solution quality. We also always use snake traversal for MatMul candidates, since it tends to improve locality across neighboring tiles. The maximum k is capped by the native granularity, and split-k compute time is scaled proportionally to k/k_{native}.

For all-pointwise chains and cones, the execution rule is simpler because pointwise operators have no reduction dimension. We therefore fix $k = 1$ and evaluate only spatial tile sizes. From the chosen granularity we can compute the required input/output tile size and reject any candidate that would exceed fast-memory capacity. This makes pointwise fusion both easy to validate and efficient to search.

2.4 Tile simulator

The tile simulator is the core of the cost model. Every candidate subgraph is validated by simulation rather than heuristic estimation. The simulator checks granularity validity, and computes the peak working set over external inputs, boundary outputs and retained tensors. A candidate is considered feasible only if all tiles can be executed without exceeding the fast memory limit. This should ensure that the solver never accepts an out-of-memory schedule.

2.5 Retention policy

Because fusions are not always profitable or even feasible, retention is important to avoid redundant data traffic. Our retention policy is conservative. We only retain current subgraph outputs, only when the tensor fits in fast memory, and only when the future dependency pattern is safe. In addition, when a tensor is retained in the fast memory instead of storing to fast memory, the next scheduling step must consume all of its remaining uses; otherwise that state expansion is rejected.

3 Discussion

Our method prioritizes correctness over more speculative fusion. In particular, we do not aggressively optimize ambiguous mixed cases unless their fusion rule is clearly defined. This conservative choice makes the solver efficient and reliable, while still capturing the most valuable optimizations in the benchmarks.

4 Experimental Results

We evaluated the compiled `mlsys` binary on the five publicly released benchmark problems. All experiments completed successfully within the specified timeouts. The reference results are summarized below.

Table 1: Latency Results on Public Benchmarks

Benchmark	Total Latency
mlsys-2026-1	471,501
mlsys-2026-5	961,195
mlsys-2026-9	167,531,000
mlsys-2026-13	166,409,000
mlsys-2026-17	46,386,500